

任务 1 搭建推拉流服务器

1、租一个linux云服务器

实例规格 

CPU - 2核 内存 - 2GB

系统盘 - SSD云硬盘 50GB [管理快照](#)

流量包 - 300GB/月 (带宽: 4Mbps)

镜像名称



Ubuntu [重装系统](#) [制作镜像](#)

镜像类型

系统镜像

操作系统

Ubuntu Server 18.04.1 LTS 64bit

2、使用FinalShell或其他工具登录到服务器

3、使用nginx搭建推拉流服务器

Nginx 是一款高性能的HTTP和反向代理服务器，也是一个IMAP/POP3/SMTP代理服务器。能够处理高达 50,000 个并发连接数，同时其配置文件非常直观。Nginx 通常被用作负载均衡器和HTTP缓存工具，并在许多高负载网站（例如Netflix, Airbnb等）中使用。

(1) 创建文件夹

```
mkdir nginx
cd nginx/
```

(2) 下载需要的软件包并解压

build-essential: 包含了编译Ubuntu软件所需的基本编译器工具，比如 gcc 和 g++。

libpcre3, libpcre3-dev: 这些是用于处理正则表达式的库，nginx 使用它解析正则表达式。

libssl-dev: 包含用于安全套接字层工具链的开发库，nginx 使用它支持 HTTPS。

zlib: 提供压缩算法，nginx 用它来进行gzip压缩。

openssl: 提供加密功能，nginx 用来支持 HTTPS。

nginx-rtmp-module: 一个支持RTMP直播流的nginx模块，可以用于直播视频流的管理。

```

sudo apt-get install build-essential libpcre3 libpcre3-dev libssl-dev
wget http://nginx.org/download/nginx-1.10.3.tar.gz
tar xvf nginx-1.10.3.tar.gz
wget http://zlib.net/zlib-1.3.1.tar.gz
tar xvf zlib-1.3.1.tar.gz
wget http://downloads.sourceforge.net/project/pcre/pcre/8.35/pcre-8.35.tar.gz
tar xvf pcre-8.35.tar.gz
wget https://www.openssl.org/source/openssl-1.0.2k.tar.gz
tar xvf openssl-1.0.2k.tar.gz
wget https://github.com/arut/nginx-rtmp-module/archive/master.zip
tar xvf nginx-rtmp-module-master.tar.gz

```

文件夹里包含如下（文件夹位置可能不一样，可以自行定义）

```

root@VM-8-16-ubuntu:/home/ubuntu/nginx# ls
master.zip  nginx-1.10.3  nginx-1.10.3.tar.gz  nginx-rtmp-module-master  openssl-1.0.2k  openssl-1.0.2k.tar.gz  pcre-8.35  pcre-8.35.tar.gz  zlib-1.3.1  zlib-1.3.1.tar.gz

```

(3) 配置nginx，生成Makefile文件并修改

```

--prefix=/usr/local/nginx: 指定nginx安装的目录。

--with-debug: 允许nginx生成调试信息，有助于调试。

--with-pcre=../pcre-8.35: 指定pcre的路径，用于编译nginx。

--with-zlib=../zlib-1.3.1: 指定zlib路径。

--with-openssl=../openssl-1.0.2k: 指定openssl路径。

--add-module=../nginx-rtmp-module-master: 添加nginx RTMP模块，支持视频流传输。

```

```

cd nginx-1.10.3/
./configure --prefix=/usr/local/nginx --with-debug --with-pcre=../pcre-8.35 --
with-zlib=../zlib-1.3.1 --with-openssl=../openssl-1.0.2k --add-module=../nginx-
rtmp-module-master
vim objs/Makefile

```

`-werror` 的作用是将所有的警告（warnings）转换为错误，**把它删了**

```

CC = cc
CFLAGS = -pipe -O2 -W -Wall -Wpointer-arith -Wno-unused -Werror -g -I../nginx-rtmp-module-master
CPP = cc -E
LINK = $(CC)

ALL_INCS = -I src/core \
           -I src/event \
           -I src/event/modules \
           -I src/os/unix \
           -I ../pcre-8.40 \
           -I ../openssl-1.0.2k/openssl/include \
           -I ../zlib-1.2.11 \
           -I objs \
-- INSERT --

```



(4) 编译安装

```
make & make install
```

(5) 设置开机启动, 设置 Nginx 作为服务在 Ubuntu 系统上的启动和管理

```
wget http://raw.githubusercontent.com/JasonGiedymin/nginx-init-ubuntu/master/nginx -O
/etc/init.d/nginx
chmod +x /etc/init.d/nginx
update-rc.d nginx defaults
```

(6) 启动nginx服务 (下面三个命令是启动, 停止和重启) 之后修改配置文件后要重启

```
service nginx start
service nginx stop
service nginx restart
```

(7) 找到/usr/local/nginx/conf/nginx.conf文件, 修改为:

```
user root;

worker_processes 2;    #CPU核数

error_log logs/error.log;
#error_log logs/error.log notice;
error_log logs/error.log info;

#pid logs/nginx.pid;

events {
    worker_connections 1024;
}

rtmp_auto_push on;

#RTMP服务
rtmp {
    server {
        listen 9090;
        application live {

            live on;    #开启实时

            record all;
            record_unique on;
            record_path "/usr/local/rtmp_video";#视频缓存的路径
            record_suffix -%Y-%m-%d-%H_%M_%S.flv;

        }
    }
}

#HTTP服务
http {
    include mime.types;
    default_type application/octet-stream;
    sendfile on;
    keepalive_timeout 65;
```

```

server {
    listen      9092;
    server_name localhost;
    location / {
        root /usr/local/rtmp_video; #指定哪个目录作为Http文件服务器的根目录
        autoindex on;#设置允许列出整个目录
        autoindex_exact_size off;#on显示出文件的确切大小，单位是bytes;off显示大概大小
        autoindex_localtime on;#显示文件的服务器时间
        charset utf-8;#防止文件乱码显示，如果用utf-8还是乱码，就改成gbk试试
    }
}

server{
    listen      9091;
    server_name localhost;
    location / {
        root html;
        index index.html index.htm;
    }
}
}

```

(7) 重启nginx服务，这时候访问9091端口，就会显示如图，说明配置成功了。

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

(8) 测试推拉流

手机端推流工具：EasyRTMP

PC端推流工具：OBS studio

拉流工具：VLC

推流地址：rtmp://IP地址:9090/live/[任意值] 拉流地址相同

访问 http://服务器IP地址:9092 就可以看到推流的视频记录

Index of /

../		
-1717769582-2024-06-07-22_13_02.flv	07-Jun-2024 22:13	6M
1-1717334076-2024-06-02-21_14_36.flv	02-Jun-2024 21:14	2M
1-1717334121-2024-06-02-21_15_21.flv	02-Jun-2024 21:15	2M
1-1717334243-2024-06-02-21_17_23.flv	02-Jun-2024 21:18	22M
1-1717334523-2024-06-02-21_22_03.flv	02-Jun-2024 21:22	5M
1-1717335091-2024-06-02-21_31_31.flv	02-Jun-2024 21:32	13M

任务2 配置Python环境

- 1、安装conda
- 2、配置 Python 运行环境并安装依赖

```
conda create -n env-name python=3.7
pip install -r requirements.txt
```

```
#requirements.txt
numpy~=1.21.6
opencv-python~=3.4.9.33
dlib~=19.24.0
flask~=2.1.1
pillow~=9.5.0
```

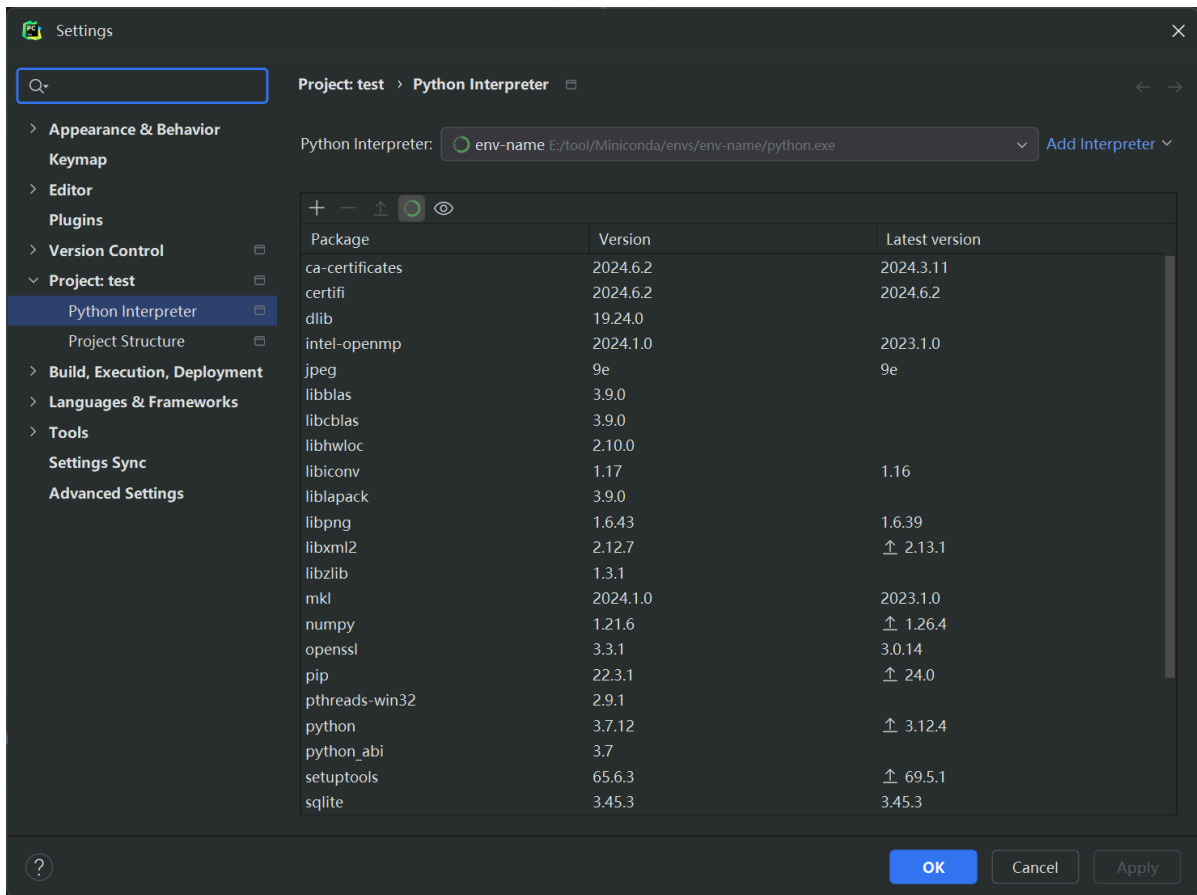
安装dlib时可能会报错

```
Failed to build dlib
ERROR: Could not build wheels for dlib, which is required to install pyproject.toml-based projects
```

可以执行下面命令下载

```
conda install -c https://conda.anaconda.org/conda-forge dlib
```

在pycharm中使用对应的环境



任务3 配置好前端环境

以vue前端框架为例：

[Vue安装及环境配置、开发工具 vue需要安装环境吗-CSDN博客](#)

[Vue安装环境最全教程，傻瓜式安装-CSDN博客](#)

任务4 进行实时数据传输

1、使用flask框架搭建一个简单的后端进行拉流，后续也在这里处理数据流

```
from flask import Flask, jsonify, Response
import cv2

app = Flask(__name__)

def process(stream_url):
    cap = cv2.VideoCapture(stream_url)
    # 设置分辨率为480x480 要不然延时很严重
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 480)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    frame_skip = 5 # 跳过的帧数
    frame_count = 0
    while True:
        success, frame = cap.read()
        if not success:
```

```

        break
    if frame_count % frame_skip == 0:
        # 编码图像
        ret, buffer = cv2.imencode('.jpg', frame,
[int(cv2.IMWRITE_JPEG_QUALITY), 70])
        frame = buffer.tobytes()
        yield (b'--frame\r\n'
            b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n') # 构造
响应体

        frame_count += 1

@app.route('/video_feed/<stream_id>')
def video_feed(stream_id):
    try:
        # 改成服务器ip地址
        stream_url = f'rtmp://服务器IP:9090/live/{stream_id}'
        # 以 multipart/x-mixed-replace 格式返回数据流
        return Response(process(stream_url),
            mimetype='multipart/x-mixed-replace; boundary=frame')
    except Exception as e:
        app.logger.error(f'Failed to stream video for ID {stream_id}: {str(e)}')
        return jsonify({'error': 'Failed to process video stream'}), 500

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False, threaded=True)

```

2、写一个简单的前端页面从后端获取并展示数据流

```

<template>
  <div class="video-stream-viewer">
    <select v-model="selectedStream">
      <option value="1">Stream 1</option>
      <option value="2">Stream 2</option>
      <option value="3">Stream 3</option>
    </select>
    
  </div>
</template>

<script>
export default {
  name: 'VideoStreamViewer',
  data() {
    return {
      selectedStream: '1',
      videoFeedBaseUrl: 'http://127.0.0.1:5000/video_feed/'
    };
  },
  computed: {
    videoFeedUrl() {
      return `${this.videoFeedBaseUrl}${this.selectedStream}`;
    }
  },
  methods: {

```

```

    handleImageError() {
        console.error('Failed to load the video stream image.');
```

3、运行

现在就可以通过rtmp://服务器IP:9090/live/1推流查看效果了！

你可以同时使用手机和电脑屏幕使用rtmp://服务器IP:9090/live/1和rtmp://服务器IP:9090/live/2进行推流，在前端进行切换

任务5 人脸识别功能

1、初始化 dlib 的人脸检测器和人脸识别模型

detector: dlib 提供的人脸检测器，用于在图像中定位人脸。

sp: 形状预测器，用于在检测到的人脸上识别68个关键点，用于后续的特征提取和人脸识别。

facerec: 基于 ResNet 的人脸识别模型，生成128维的人脸编码，这些编码用于比较和识别不同的人脸。

下面两个模型可以在[Index of /files \(dlib.net\)](https://dlib.net/files/)进行下载

```

detector = dlib.get_frontal_face_detector()
sp = dlib.shape_predictor('./dat/shape_predictor_68_face_landmarks.dat')
facerec =
dlib.face_recognition_model_v1('./dat/dlib_face_recognition_resnet_model_v1.dat')
```

2、加载已注册的人脸编码，在这里我使用json文件保存已经注册的人脸编码，在项目目录创建registered_faces.json文件，需要加上{}，不能是完全空的。

```

registered_faces_file = 'registered_faces.json'

def load_registered_faces():
    if os.path.exists(registered_faces_file):
        with open(registered_faces_file, 'r') as file:
            return json.load(file)
    return {}

registered_faces = load_registered_faces()
```

3、实现人脸注册：student_id是数据库要存储的主键，文件中存储student_id和image（图像编码）的键值对

```

@app.route('/face-register', methods=['POST'])
def face_register():
    data = request.get_json()
    student_id = data['student_id']
    image_data = data['image']
```

```

# 解码并转换为 PIL 图像, 然后转为 NumPy 数组以便 dlib 处理
image = Image.open(BytesIO(base64.b64decode(image_data)))
image_np = np.array(image)

# 使用 detector 检测图像中的人脸, 然后通过 sp 获取关键点, facerec 生成人脸编码。
faces = detector(image_np, 1)
if len(faces) != 1:
    return jsonify({'error': 'No face or multiple faces detected'}), 400

shape = sp(image_np, faces[0])
face_descriptor = facerec.compute_face_descriptor(image_np, shape)
face_descriptor_list = [x for x in face_descriptor]

# 人脸编码存储在 JSON 文件中, 与学生 ID 关联
registered_faces[student_id] = face_descriptor_list
with open(registered_faces_file, 'w') as file:
    json.dump(registered_faces, file)

return jsonify({'message': 'Face registered successfully'})

```

导入需要的库

```

import json
import os

from flask import Flask, request, jsonify, Response
import base64
from io import BytesIO
from PIL import Image
import numpy as np
import cv2
import face_recognition
import dlib

```

在前端写好页面, 从摄像头录入人脸

```

<template>
  <div class="registration-container">
    <h2>人脸录入</h2>
    <div class="video-container">
      <video ref="video" width="640" height="480" autoplay></video>
      <button @click="captureAndRegister">录入人脸</button>
    </div>
  </div>
</template>

<script setup>
  import { ref, onMounted } from 'vue';
  import { useRoute, useRouter } from 'vue-router';
  import axios from 'axios';

  const route = useRoute();
  const router = useRouter();
  const video = ref(null);

```

```

const studentId = route.query.id;

onMounted(async () => {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({ video: true
  });
    if (video.value) {
      video.value.srcObject = stream;
    }
  } catch (error) {
    console.error('Error accessing the camera:', error);
  }
});

//录制当前视频帧，将图像转换为 JPEG 的 base64 编码，发送到后端进行注册
async function captureAndRegister() {
  if (!video.value) return;

  const canvas = document.createElement('canvas');
  canvas.width = video.value.videoWidth;
  canvas.height = video.value.videoHeight;
  const context = canvas.getContext('2d');
  context.drawImage(video.value, 0, 0, canvas.width, canvas.height);
  const imageBase64 = canvas.toDataURL('image/jpeg').split(',')[1];

  try {
    //这里在vue.config.js中设置了HTTP代理,实际上就是在访问
    http://127.0.0.1:5000/face-register
    const response = await axios.post('/python/face-register', {
      student_id: studentId,
      image: imageBase64
    }, {
      headers: {
        'Content-Type': 'application/json'
      }
    });

    if (response.status === 200) {
      alert('人脸录入成功! ');
      router.replace('/index');
    } else {
      alert('人脸录入失败, 请重试! ');
    }
  } catch (error) {
    console.error('Error during face registration:', error.message);
    alert('人脸录入失败, 请重试! ');
  }
}
</script>

```

4、进行人脸检测

```

def gen_frames(stream_url):
    cap = cv2.VideoCapture(stream_url)
    # 设置分辨率为480*480

```

```

cap.set(cv2.CAP_PROP_FRAME_WIDTH, 480)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
frame_skip = 5 # 跳过的帧数
frame_count = 0
while True:
    success, frame = cap.read()
    if not success:
        break
    if frame_count % frame_skip == 0:
        # 1. 转换为灰度图像
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        # 2. 检测人脸
        faces = detector(gray, 1)
        for face in faces:
            # 获取人脸关键点
            shape = sp(frame, face)
            # 计算人脸的128维编码
            face_encoding = np.array(facerec.compute_face_descriptor(frame,
shape))

            # 比较捕获的人脸与已注册人脸库中的编码，以判断是否为已知人脸
            matches =
face_recognition.compare_faces(list(registered_faces.values()), face_encoding,
tolerance=0.4)

            name = "Stranger"
            color = (0, 0, 255) # 默认红色标记陌生人

            if True in matches:
                first_match_index = matches.index(True)
                student_id = list(registered_faces.keys())[first_match_index]
                name = student_id
                color = (0, 255, 0) # 绿色标记已注册人脸

            # 在人脸周围绘制矩形框
            cv2.rectangle(frame, (face.left(), face.top()), (face.right(),
face.bottom()), color, 2)
            # 添加文本标签
            cv2.putText(frame, name, (face.left(), face.top() - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.9, color, 2)

            # 3. 编码图像
            ret, buffer = cv2.imencode('.jpg', frame,
[int(cv2.IMWRITE_JPEG_QUALITY), 70])
            frame = buffer.tobytes()
            yield (b'--frame\r\n'
                b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n') # 构造
响应体

            frame_count += 1

@app.route('/video_feed/<stream_id>')
def video_feed(stream_id):
    try:
        stream_url = f'rtmp://127.0.0.1:9090/live/{stream_id}'
        return Response(gen_frames(stream_url),
                        mimetype='multipart/x-mixed-replace; boundary=frame')
    except Exception as e:
        app.logger.error(f"Failed to stream video for ID {stream_id}: {str(e)}")

```

```
return jsonify({'error': 'Failed to process video stream'}), 500
```

现在前端展示的就是进行人脸检测后的画面了！

任务6 搭建后端进行用户管理

1、搭建项目（示例为SpringBoot+MyBatis+MySQL）

2、配置swagger

导入依赖：

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>2.1.0</version>
</dependency>

<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-api</artifactId>
    <version>2.1.0</version>
</dependency>
```

编写配置类

```
@Configuration
public class SwaggerConfig {

    @Bean
    public GroupedOpenApi publicApi() {
        return GroupedOpenApi.builder()
            .group("spring")
            .pathsToMatch("/**")
            .build();
    }
}
```

通过<http://localhost:8080/swagger-ui.html>查看API文档